Resumen de la Metodología Agéntica

Nombre: Arquitectura Multi-Agente Actor-Crítico (*Actor-Critic Multi-Agent Architecture* / Agentic CI-CD Loop).

Resumen: Es un sistema autónomo de desarrollo de software donde un sub-agente implementador escribe código y un sub-agente auditor independiente lo evalúa contra un estándar objetivo, iterando de forma autónoma mediante retroalimentación estructurada hasta cumplir el baremo exigido, sin intervención humana.

Documentación Detallada de la Metodología

1. Propósito y Filosofía

El objetivo de esta metodología es eliminar el bottleneck humano en el ciclo de vida del desarrollo de software (SDLC) al automatizar no solo la generación de código, sino también su revisión, prueba y corrección. Se basa en el principio de separación de responsabilidades y validación cruzada, previniendo el sesgo de confirmación (evitando que el mismo LLM que generó el código valide su propia infalibilidad).

2. Arquitectura de Roles

El sistema requiere una estructura jerárquica compuesta por tres entidades lógicas independientes:

- Agente Orquestador (Product Owner / Tech Lead): Es el punto de contacto con el usuario. Recibe el requerimiento, traduce las necesidades del usuario en un "baremo" (criterios de aceptación estrictos y medibles), descompone la tarea en módulos accionables y coordina el flujo entre el Actor y el Crítico.
- Sub-Agente Implementador (Actor / Developer): Su única función es recibir la tarea y el estándar, y escribir o modificar el código para cumplir el objetivo. No decide si el código está terminado; simplemente responde a los requerimientos y a la retroalimentación del Crítico.
- Sub-Agente Auditor (Crítico / Reviewer): Totalmente independiente del Implementador. Su rol es ejecutar el "Gauntlet" (batería de validaciones). Analiza el código, ejecuta pruebas estáticas y dinámicas, y emite un veredicto objetivo. Tiene capacidad de ejecución en consola (Sandbox).

3. El Ciclo de Vida Autónomo (El Bucle de Iteración)

El flujo de trabajo se ejecuta de la siguiente manera:

Fase 1: Ingestión y Definición del Baremo

El usuario emite una petición (ej. "Crear una función de cálculo de impuestos"). El Orquestador define el baremo:

- *Criterio 1:* Tipado estricto (sin uso de `any` o `dynamic`).
- *Criterio 2:* Cobertura de pruebas unitarias > 90%.
- *Criterio 3:* Manejo explícito de valores nulos lanzando excepción `ValueError`.

Fase 2: Delegación e Implementación

El Orquestador envía la tarea al Sub-agente Implementador. Este genera el código fuente y opcionalmente los tests unitarios.

Fase 3: Gauntlet de Validación Cruzada

El Orquestador toma el código del Implementador y se lo entrega al Sub-agente Auditor. El Implementador no participa en esta fase. El Auditor ejecuta el gauntlet:

1. Validación Estática: El Auditor usa herramientas CLI (ej. `grep` para buscar patrones prohibidos, `eslint`, `pylint`).
2. Validación Dinámica: El Auditor compila el código y ejecuta los tests en un entorno aislado (`npm run test`, `pytest`).
3. Validación Lógica: El Auditor simula entradas maliciosas o edge-cases (fuzzing) para intentar romper la función.

Fase 4: Resolución y Veredicto

El Sub-agente Auditor emite su respuesta estructurada (normalmente en formato JSON para que el Orquestador pueda procesarlo programáticamente):

```json
{
 "status": "FAIL",
```

```
 "errors": [
   "Validación estática: Se encontró tipo 'any' en línea 12.",
   "Validación dinámica: Test 'null_input' falló con NullReferenceException."
 ],
 "feedback_for_actor": "Reemplazar el tipo 'any' por 'number'. Añadir un bloque condicional al inicio de la función para lanzar 'ValueError' si el input es nulo."
}
```

Fase 5: Auto-corrección (Iteración)

Si el estado es `FAIL`, el Orquestador toma el campo `feedback_for_actor` y se lo devuelve al Sub-agente Implementador junto con el código previo. El Implementador reescribe el código y lo devuelve al Orquestador. El proceso vuelve a la Fase 3.

Fase 6: Cierre Autónomo

El bucle de iteración se repite hasta que el Sub-agente Auditor emite un `status: "PASS"`. En ese momento, el Orquestador detiene la iteración, consolida el código y lo entrega al usuario o lo fusiona en la rama de desarrollo correspondiente.

4. Lógica de Control y Medidas de Seguridad

Para que este sistema sea viable en producción, el controlador del bucle (el código que gestiona a los LLMs) debe incluir medidas de seguridad:

- Límite de Intentos (Max Iterations): Si el Actor no logra pasar el Gauntlet después de *N* iteraciones (ej. 5 intentos), el Orquestador debe pausar el proceso autónomo y escalar el problema a un ingeniero humano. Esto evita loops infinitos por alucinaciones persistentes.
- Sandboxing: El Sub-agente Auditor debe tener permisos de ejecución de código solo en un entorno aislado (Docker, WebAssembly, etc.) para evitar que el código generado dañe el sistema operativo del servidor.

5. Requisitos Técnicos de Implementación

Para desplegar esta metodología, se recomiendan frameworks de agentes como LangGraph, CrewAI o AutoGen, los cuales permiten:

1. Definir roles (system prompts) distintos para el Actor y el Crítico.
2. Crear grafos de estado (State Machines) que dicten el flujo del bucle.
3. Proveer herramientas (Tools) al agente Crítico, como acceso a una terminal de comandos para ejecutar scripts de testing y análisis (`grep`, `npm test`, etc.).